From ***plain Specifications to MeTTa/PeTTa and Rholang: A Typed Atomspace IR Proposal

OmegaClaw / ZeroBot working notes for Ben Goertzel

2026-06-29

Abstract

This note reviews a short design discussion about whether software specifications written in the ***plain style can be mapped into MeTTa, initially PeTTa as a MeTTa dialect, and later into Rholang and MeTTa-IL. The conclusion is positive but conditional: the viable target is not a magic compiler from arbitrary English to correct code, but an assisted semantic lowering pipeline from structured specifications to an inspectable typed logical intermediate representation. The preferred intermediate representation is an Atomspace-style typed logical graph, designed as a logical mirror of MeTTa and constrained by Curry-Howard and OSLF-style correspondences. This IR should make MeTTa/PeTTa generation relatively natural, keep Rholang generation grounded in explicit process semantics, and open the door to later PLN-based analysis of requirements, contradictions, underspecification, test coverage, and confidence.

1 Provenance and purpose

This document summarizes and expands a 2026-06-29 discussion between Ben Goertzel, ProtomegaTron/OmegaClaw, and ZeroBot/OpenClaw about using `***plain` software specifications as input to a semantic toolchain. The motivating links were the ***plain documentation at `https://plainlang.org/docs/` and the Plain GitHub organization at `https://github.com/plainlang`.

The immediate question was whether one could develop a tool that maps plain software specifications into MeTTa, using PeTTa as an initial target dialect, and then into Rholang as well. A later extension would target MeTTa-IL. Ben then suggested that an Atomspace-based logic representation might be the right intermediate representation, for two reasons:

(1) if the logic representation is a good logical mirror of MeTTa, in the sense suggested by Curry-Howard and Meredith/Stay OSLF-style type mappings, then mapping the IR to MeTTa code becomes natural rather than ad hoc;

(2) such an IR opens the door to later PLN-based analysis of specifications.

The purpose of this note is to turn that conversational design intuition into a more detailed information package for collaborators who may suggest designs, identify risks, or help build a prototype.

2 What ***plain contributes

The public ***plain documentation presents ***plain as a specification language for writing software requirements in a clear structured format. A file has optional YAML frontmatter followed by standardized sections. The core sections include:

- `***definitions***`, declaring named concepts such as `:App:`, `:User:`, `:Task:`, and their attributes or constraints;
- `***implementation reqs***`, giving non-functional and implementation-oriented constraints such as language, framework, architecture, naming conventions, and environment;
- `***test reqs***`, specifying how conformance tests should be structured and executed;
- `***functional specs***`, listing small, testable functional requirements; and
- nested `***acceptance tests***`, giving observable criteria under individual functional requirements.

Important properties for this proposal are:

1. concepts are explicitly named using a lightweight notation such as `:Task:`;
2. definitions can refer to other definitions;
3. attributes and constraints can be represented as nested bullets;
4. functional requirements are already encouraged to be small and testable;
5. acceptance tests are explicitly linked to requirements; and
6. imported modules and template libraries can contribute reusable definitions and constraints.

These features make ***plain a promising human-readable front end for semantic software specification. The crucial limitation is that many bodies are still natural language. Therefore, the MVP should avoid pretending that arbitrary English has been fully formalized.

3 Main viability claim

Proposition 1 (Viability as assisted semantic lowering)**.** *A toolchain from ***plain to MeTTa/PeTTa and Rholang is viable if it is framed as an assisted semantic lowering pipeline with an inspectable typed logical IR, not as an automatic compiler from arbitrary natural language into correct executable systems.*

The proposed pipeline is:

```
.plain source
  -> parsed Plain AST
  -> typed Atomspace / logical Spec IR
  -> MeTTa / PeTTa emitter
  -> PLN and consistency-analysis passes
  -> Rholang process-model emitter
  -> later MeTTa-IL emitter
```

The MVP can deliberately keep natural-language interpretation shallow. It can parse explicit section structure, concept names, bullets, nesting, provenance, and some constrained patterns, while preserving ambiguous text as quoted claims or obligations for later human/LLM refinement.

4 Why an Atomspace-style logical IR is attractive

Design Principle 1 (Typed logical mirror)**.** *The central IR should be a typed logical graph that mirrors the relevant logical and type-theoretic structure of MeTTa closely enough that MeTTa emission is a principled rendering, not a backend-specific rewrite trick.*

An Atomspace-style IR is attractive because it can represent concepts, relations, propositions, procedures, tests, and truth/confidence annotations in a single hypergraph-like substrate. It can also preserve provenance and partial formalization status without forcing premature executable precision.

Definition 1 (Spec atom)**.** *A spec atom is a typed IR element*

$$a = \langle id, kind, type, args, provenance, status, truth \rangle$$

where kind distinguishes concepts, predicates, links, requirements, processes, tests, and annotations; type records the logical or computational type; args are outgoing targets; provenance links back to the source file, section, bullet, and textual span; status records whether the atom is parsed, inferred, reviewed, rejected, or executable; and truth can later store PLN-style strength and confidence.

Definition 2 (Plain-to-IR lowering)**.** *A Plain-to-IR lowering function is a partial translation*

$$L : PlainAST \to (G, W)$$

where G is a typed Atomspace-style graph and W is a set of warnings, ambiguities, unresolved references, and human-review obligations.

The word "partial" matters. The first version should translate what is structural and typed, preserve what is natural-language and ambiguous, and make uncertainty visible.

5 OSLF, Curry-Howard, and the type layer

Ben's key refinement was that even without full English-to-logic parsing, the system should get type relationships right so that OSLF-style correspondences can work later. This suggests that the type layer is not decorative metadata; it is the spine of the architecture.

Design Principle 2 (Types before prose semantics)**.** *For the MVP, prioritize a correct concept/type/proposition layer over ambitious natural-language semantic parsing. Natural language can remain attached as quoted evidence, but type relationships should be explicit, checkable, and mapped coherently into MeTTa/PeTTa.*

A useful three-level distinction is:

1. **Concept layer.** Plain concepts such as `:Task:`, `:User:`, `:TaskList:`, `:Implementation:`, and `:ConformanceTests:`.
2. **Logical type graph.** Atomspace-like nodes and links expressing inheritance, instancehood, products, functions, predicates, propositions, obligations, and tests.
3. **Operational rendering.** MeTTa/PeTTa declarations and rules, Rholang processes and channels, and later MeTTa-IL constructs.

The Curry-Howard intuition is that propositions, types, proofs, programs, and process contracts should be aligned rather than represented in incompatible layers. A functional spec can be treated as an obligation/proposition whose realizer is code and whose evidence includes acceptance-test traces. A concept can be represented both as a type and as an object of reasoning. A test can be a claim about observable behavior and also a process that attempts to produce witnesses or counterexamples.

6 Candidate IR vocabulary

The following table gives an initial vocabulary. The exact Atom names are not final; the important issue is the separation of roles.

Plain element	IR role	Possible MeTTa/PeTTa rendering
`:Concept:` definition	Concept/type declaration	`(: Concept Type)` or domain-specific type atom
Concept reference	Reference edge, possibly typed	`(refers-to Req Concept)`
Nested attribute	Attribute predicate or dependent field	`(has-attribute Task Name Required)`
Implementation requirement	Non-functional constraint / realization hint	`(implementation-constraint Req Python)`
Test requirement	Conformance-test policy	`(test-policy App Unittest)`
Functional spec bullet	Requirement proposition / obligation	`(: FS1 Requirement)` and `(requires FS1 ...)`
Acceptance test	Observable evidence condition	`(acceptance-test FS1 AT1)`
Import/template	Module dependency and namespace source	`(imports Spec Module)`
Natural-language residue	Quoted claim needing interpretation	`(source-text FS1 "...")`

An Atomspace-like version might use nodes and links such as:

```
ConceptNode("Task")
ConceptNode("User")
InheritanceLink(ConceptNode("Task"), ConceptNode("PlainConcept"))
EvaluationLink(PredicateNode("has-attribute"),
  ListLink(ConceptNode("Task"), ConceptNode("Name"), ConceptNode("Required")))
EvaluationLink(PredicateNode("functional-spec"),
  ListLink(ConceptNode("FS3"), ConceptNode("User"),
           PredicateNode("can-add"), ConceptNode("Task")))
```

A MeTTa/PeTTa rendering of the same structure could be:

```
(: Task PlainConcept)
(: User PlainConcept)
(: TaskList PlainConcept)
(has-attribute Task Name Required)
(functional-spec FS3)
(actor FS3 User)
(action FS3 add)
```

```
(object FS3 Task)
(constraint FS3 (valid Task))
(acceptance-test FS3 AT3_1)
(source-text FS3 ":User: should be able to add :Task:...")
```

7 Worked miniature example

A small ***plain fragment might be:

```
***definitions***
- :App: is a console task manager application.
- :User: is the user of :App:.
- :Task: describes an activity that needs to be done by :User:.
  - Name - a short required description of :Task:.
- :TaskList: is a list of :Task: items.
  - Initially :TaskList: should be empty.

***implementation reqs***
- :Implementation: should be in Python.

***test reqs***
- :ConformanceTests: of :App: should be implemented using Unittest.

***functional specs***
- :User: should be able to add :Task:. Only valid :Task: items can be added.
  ***acceptance tests***
  - Adding a valid :Task: should make it appear in :TaskList:.
```

A conservative IR should produce at least:

1. a concept/type environment containing `App`, `User`, `Task`, `TaskList`, `Implementation`, and `ConformanceTests`;
2. relationship atoms such as `user-of(User, App)`, `list-of(TaskList, Task)`, `has-field(Task, Name)`;
3. an initial-state constraint `empty(TaskList)`;
4. an implementation constraint `target-language(Python)`;
5. a test policy `framework(Unittest)`;
6. a requirement atom for add-task behavior; and
7. an acceptance-test atom linked to the requirement.

A possible PeTTa-oriented representation is:

```
(: App PlainConcept)
(: User PlainConcept)
(: Task PlainConcept)
```

```
(: TaskList PlainConcept)
(: AddTask Requirement)
(: AddTaskAcceptance AcceptanceTest)

(user-of User App)
(list-of TaskList Task)
(has-field Task Name)
(required-field Task Name)
(initial-state TaskList Empty)
(target-language Implementation Python)
(test-framework ConformanceTests Unittest)

(requirement AddTask)
(actor AddTask User)
(action AddTask add)
(object AddTask Task)
(precondition AddTask (valid Task))
(postcondition AddTask (member Task TaskList))

(acceptance-test-of AddTaskAcceptance AddTask)
(observes AddTaskAcceptance (member Task TaskList))
(source-text AddTaskAcceptance
  "Adding a valid :Task: should make it appear in :TaskList:.")
```

This is intentionally not yet an executable task manager. It is a scrutable semantic skeleton from which code-generation, reasoning, and test-generation steps can proceed.

8 Mapping to MeTTa and PeTTa

The MeTTa/PeTTa path is the most direct because the proposed IR and MeTTa share a bias toward typed symbolic structures, hypergraph-like representation, and executable logic. PeTTa, as a MeTTa dialect or restricted profile for early experiments, can focus on a disciplined subset:

- concept declarations;
- type declarations;
- relation atoms;
- requirement and acceptance-test atoms;
- provenance atoms;
- simple rule templates for validation and test generation; and
- truth-value/confidence slots reserved for later PLN integration.

The first emitter should not try to synthesize full algorithms. It should emit semantic declarations and obligations. Examples of early useful outputs:

1. undefined-concept warnings;

2. duplicate-concept warnings;
3. requirement-to-test coverage maps;
4. type-consistency checks;
5. a queryable graph of concepts, attributes, and requirements;
6. PeTTa rules that identify underspecified requirements;
7. PeTTa rules that identify acceptance tests whose observable predicate is not connected to the parent requirement; and
8. human-readable roundtrip reports.

9 Mapping to Rholang

The Rholang mapping is less direct but still meaningful. Rholang is a process calculus language, so its natural semantic targets are not merely concepts and requirements but channels, messages, processes, state transitions, and observable traces.

A useful mapping discipline is:

- concepts become data schemas, channel payload types, or process roles;
- functional specs become process contracts or transition obligations;
- user actions become receives/sends on channels;
- application state becomes a collection of process-held names or stores;
- acceptance tests become trace assertions over observable behavior;
- implementation requirements become backend constraints, not semantic truths; and
- concurrency-relevant requirements become explicit channel/process structure.

For the task example, a Rholang-oriented IR might identify:

```
Role(User)
Process(App)
Channel(addTask)
Channel(taskListOut)
Message(addTask, Task)
State(TaskList)
TraceAssertion(after send(addTask, validTask),
               eventually observe(taskListOut, contains(validTask)))
```

The Rholang emitter should initially target small examples where the operational model is obvious. Generating Rholang from arbitrary business prose would be premature. However, if the Atomspace IR distinguishes state, roles, actions, messages, and observations, then Rholang generation becomes a controlled projection from a process-enriched semantic graph.

10 PLN-enabled specification analysis

The Atomspace IR also enables a later PLN analysis layer. This is not needed for the first parser, but it should influence the representation now.

Potential PLN/spec-analysis tasks include:

1. infer that a requirement is probably underspecified because it mentions an action but no object, state change, or observable acceptance test;
2. infer likely concept equivalences or near-duplicates across imported modules;
3. detect contradictions between implementation requirements;
4. estimate confidence that acceptance tests cover a requirement;
5. propagate confidence from ambiguous natural-language parses into later code-generation obligations;
6. identify requirements with no tests or tests with no requirement;
7. reason about whether a Rholang trace assertion plausibly realizes a MeTTa/PeTTa requirement proposition; and
8. support interactive clarification by asking the human for the smallest missing semantic commitment.

This motivates storing provenance, confidence, and interpretation status from the beginning.

11 Architecture proposal

A practical prototype can be organized as follows:

```
plainmetta/
  parser/
    plain_sections.py        # sections, bullets, nesting, spans
    concept_refs.py          # :Concept: references and definitions
  ir/
    atoms.py                 # typed Atomspace-like data model
    typecheck.py             # concept/type/proposition checks
    provenance.py            # source spans and status
  emitters/
    metta.py                 # MeTTa/PeTTa emission
    rholang.py               # small process-calculus examples
    reports.py               # HTML/Markdown/LaTeX inspection reports
  analysis/
    coverage.py              # requirements vs acceptance tests
    ambiguity.py             # unresolved NL obligations
    pln_stub.py              # future truth-value and inference hooks
  examples/
    task_manager.plain
    task_manager.metta
    task_manager.rho
```

12 MVP plan

A good MVP should be useful before solving natural-language understanding.

MVP 0: Structural parser and concept graph

- Parse sections, bullets, indentation, source spans, and YAML frontmatter.
- Extract concept definitions and concept references.
- Detect undefined, duplicate, and unused concepts.
- Emit a JSON or S-expression Plain AST.

MVP 1: Typed Atomspace Spec IR

- Lower concept definitions, attributes, implementation reqs, test reqs, functional specs, and acceptance tests into typed IR atoms.
- Preserve natural-language text as quoted source atoms.
- Attach provenance, status, and optional confidence.
- Add simple type checks and requirement/test coverage reports.

MVP 2: PeTTa/MeTTa emitter

- Emit stable PeTTa/MeTTa declarations for concepts, relations, requirements, and tests.
- Add simple query/rule examples for missing tests, unresolved concepts, and suspiciously complex functional specs.
- Roundtrip enough structure to make code review meaningful.

MVP 3: Rholang micro-emitter

- Select constrained specs with explicit actors, actions, objects, state, and observations.
- Generate small Rholang process skeletons and trace assertions.
- Treat this as process-model generation, not full application synthesis.

MVP 4: PLN-ready analysis hooks

- Add truth-value slots and confidence propagation conventions.
- Represent ambiguous interpretations as competing atoms with provenance.
- Prototype one or two PLN-style checks, such as likely missing acceptance tests or contradictory implementation constraints.

13 Key research and engineering questions

For collaborators, the most valuable questions are:

1. What is the smallest typed IR that is genuinely a logical mirror of the relevant MeTTa fragment?
2. How should OSLF-style type mappings be represented concretely enough to guide emission and checking?
3. Which Atomspace links and node categories should be treated as primitive in the Spec IR, and which should be macros?
4. Should PeTTa be a narrow profile of MeTTa syntax, a library convention, or a more explicit dialect with its own schema?
5. What restricted natural-language patterns are worth supporting in the MVP, and which should remain quoted obligations?
6. What is the correct semantic relation between a requirement proposition, a program realizer, and an acceptance-test trace?
7. How much process structure must be present before Rholang generation is meaningful?
8. How should uncertainty and provenance be encoded so that PLN can later reason over the graph without confusing source claims, inferred claims, decisions, and executable artifacts?

14 Risks

1. **Overclaiming automation.** The project should not be marketed as automatic formalization of arbitrary English. It is an assisted formalization and lowering tool.
2. **Premature backend commitment.** If the IR is too close to one backend, Rholang and MeTTa-IL may become awkward. The IR should be logical first, backend-renderable second.
3. **Weak type discipline.** If concept/type/proposition distinctions are fuzzy, the later OSLF/Curry-Howard story will become cosmetic rather than structural.
4. **Natural-language residue.** Quoted text must not be silently treated as a verified logical formula. Status and confidence must be explicit.
5. **Rholang mismatch.** Rholang generation requires operational process semantics. Not every requirement has enough process structure.
6. **PLN pollution.** PLN should reason over typed, provenance-aware claims, not over undifferentiated strings.

15 Recommended near-term deliverable

The recommended first deliverable is a small repository containing:

1. a parser for a useful subset of ***plain;

2. a typed Atomspace-style Spec IR schema;
3. one task-manager example from ***plain to IR to PeTTa/MeTTa;
4. one small actor/process example from IR to Rholang skeleton;
5. an inspection report showing concept graph, requirement coverage, acceptance-test links, warnings, and unresolved natural-language obligations; and
6. a short design note explaining how the type layer is intended to support OSLF/Curry-Howard correspondences and later PLN analysis.

This deliverable would be small enough to build quickly but rich enough to test the central architectural bet: that Plain's readable specification structure can be lowered into an Atomspace-style typed logical graph that naturally emits MeTTa/PeTTa and can later support Rholang, MeTTa-IL, and PLN analysis.

16 Bottom line

The strongest version of the idea is:

```
***plain
  -> human-readable structured spec
  -> typed Atomspace logical mirror
  -> PeTTa/MeTTa semantic rendering
  -> PLN-style spec analysis
  -> Rholang process realization for process-suitable fragments
  -> later MeTTa-IL lowering
```

This keeps the simple path simple: parse concepts, requirements, tests, and provenance. But it also avoids a dead-end architecture by making the IR rich enough for type-theoretic interpretation, MeTTa alignment, PLN analysis, and process-calculus generation.